Algorytmy i Struktury Danych

Autorzy: Mateusz Antecki 167829, Dawid Matelski 167930
Prowadzący: dr. Maciej Machowiak

Porównanie struktur danych: Lista jednokierunkowa vs BST vs AVL

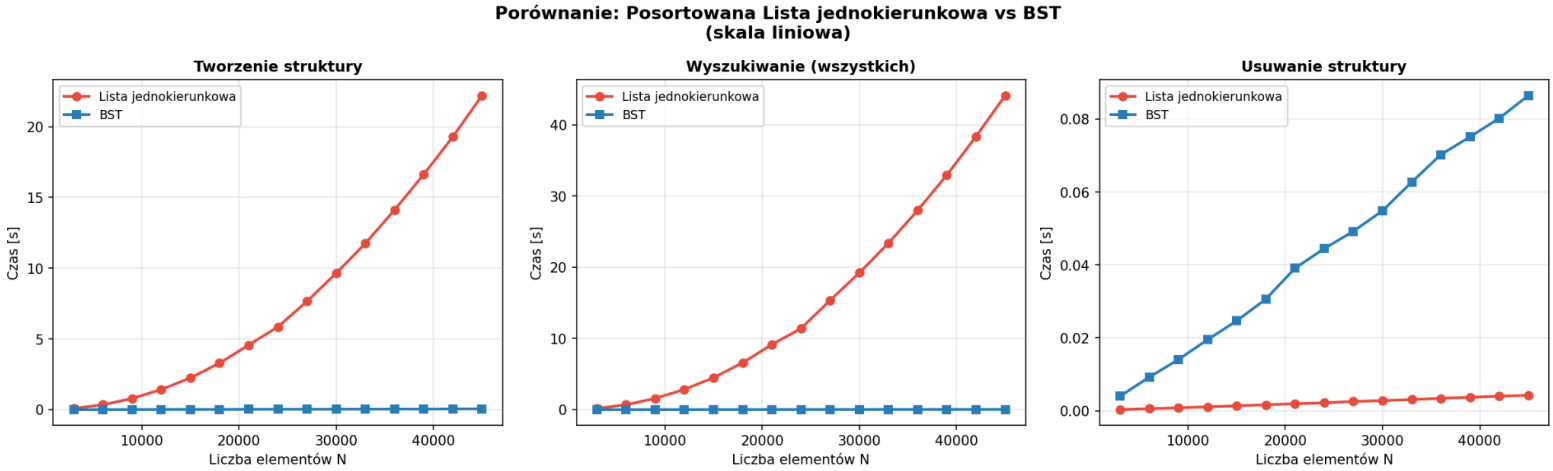
Spis Treści:

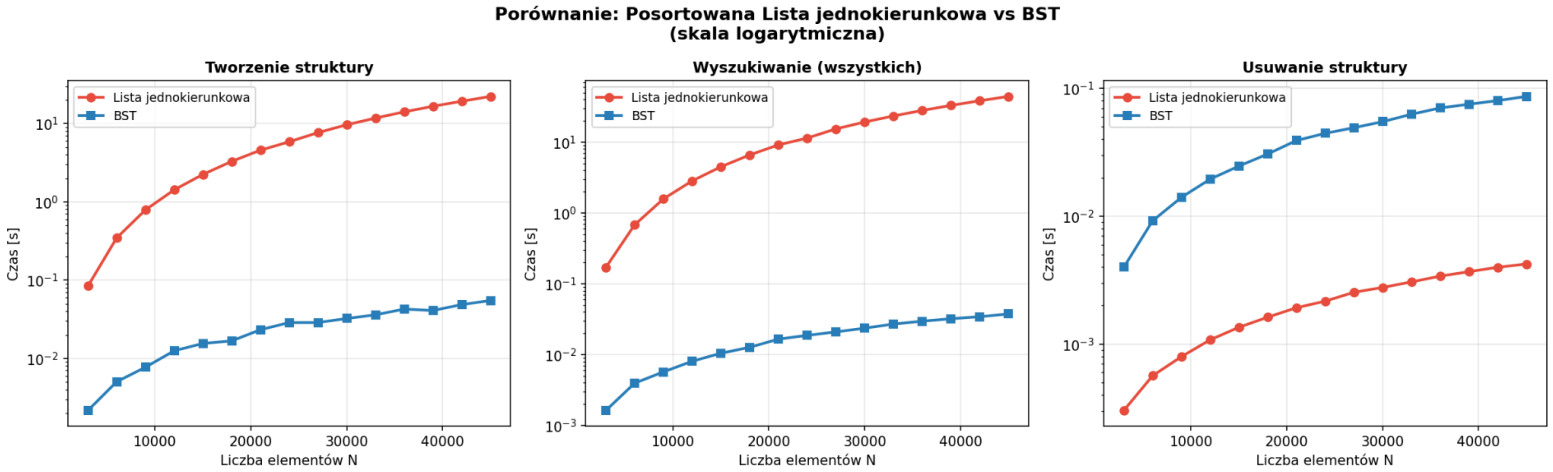
Zadanie 1 ..... **2**
Opis zadania 2
Analiza wyników 3
Złożoność obliczeniowa 3
Wnioski 4
Zadanie 2 ..... **5**
Opis zadania 5
Analiza wyników 6
Cel i skutki wyważania drzewa 6
Wnioski 6
Narzędzia pomiarowe **7**
Język i środowisko programistyczne 7
Precyzja pomiaru czasu 7
Przetwarzania i wizualizacja danych 7

Zadanie 1

Opis zadania

Na podstawie nieposortowanego ciągu niepowtarzających się liczb naturalnych zbudowano dwie struktury danych: posortowaną listę jednokierunkową i drzewo poszukiwań binarnych (BST). Dla każdej ze struktur zmierzono: czas tworzenia struktury, sumaryczny czas wyszukiwania wszystkich elementów jeden po drugim, czas usuwania struktury.





Analiza wyników

Wykresy

Wyniki pomiarów przedstawiono na wykresach powyżej, zarówno w skali liniowej, jak i logarytmicznej, co pozwala lepiej zobrazować różnice między strukturami.

Tworzenie struktury i wyszukiwanie

Na wykresach wyraźnie widać różnicę w zachowaniu obu struktur wraz ze wzrostem liczby danych. Posortowana lista jednokierunkowa bardzo szybko traci wydajność, ponieważ przy 30 tysiącach elementów czas operacji liczony jest już w sekundach, kiedy dla drzewa BST są to części milisekundy. Wynika to z faktu, że w liście każdy nowy element wymusza przeszukiwanie jej od początku, co przy dużej skali powoduje ogromne opóźnienia. Dzięki swojej rozgałęzionej strukturze, drzewo BST pozwala za każdym krokiem odrzucić znaczną część danych, co znacząco przyspiesza pracę.

Usuwanie struktury

Przy operacji usuwania lista wypada lepiej od drzewa BST, można to wyjaśnić prostotą operacji usuwania. Usuwanie zawsze pierwszego elementu to tylko kwestia przepięcia jednego wskaźnika. W przypadku drzewa ten proces jest bardziej złożony obliczeniowo, co powoduje nieco wyższe czasy operacji.

Złożoność obliczeniowa

Operacja	Lista (śr.)	BST(śr.)	BST (worst case)
Wstawianie	O(n)	O(log n)	O(n)
Wyszukiwanie	O(n)	O(log n)	O(n)
Usuwanie (pierwsze)	O(1)	O(log n)	O(n)

Wnioski

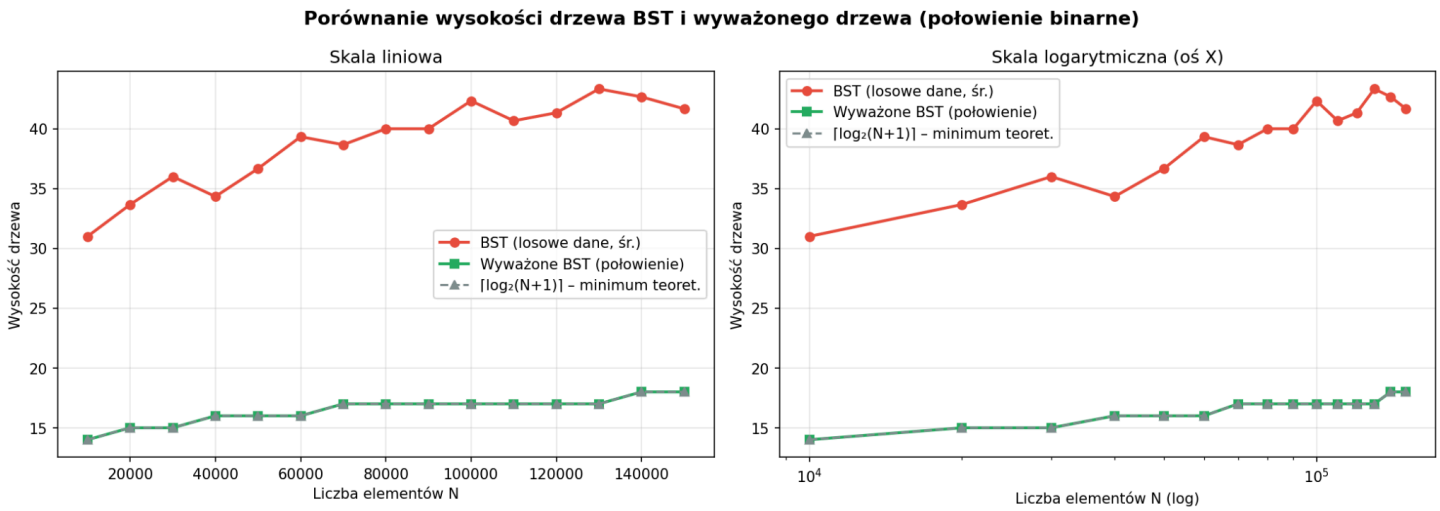
Wybór struktury danych zależy od charakteru wykonywanych operacji:

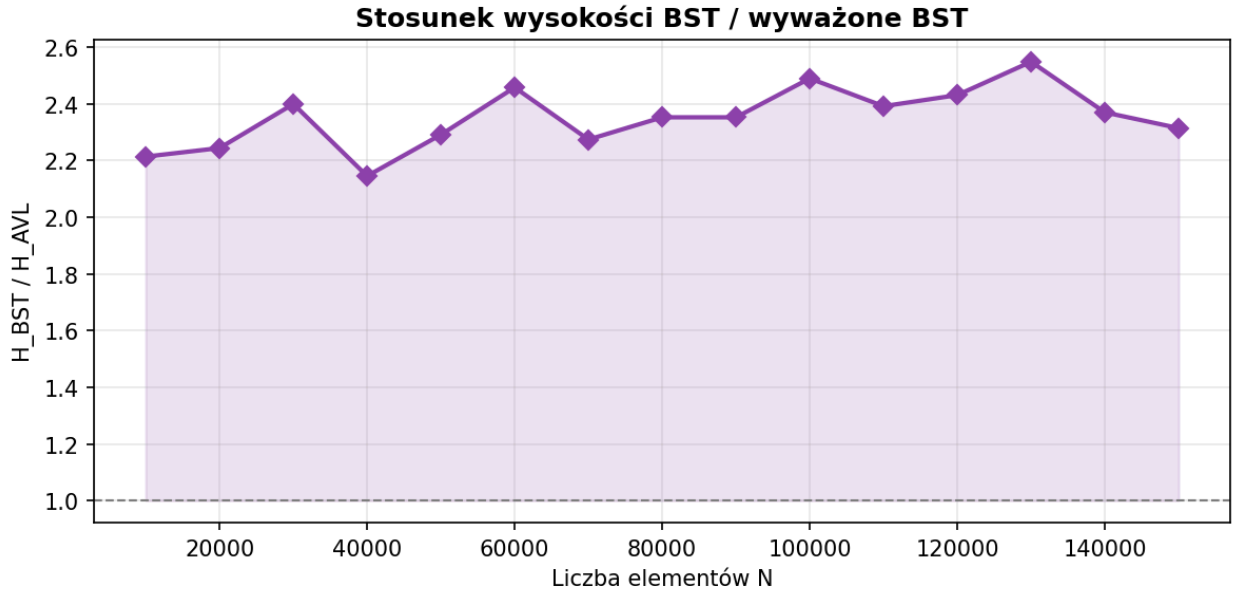
- Lista jednokierunkowa sprawdza się jedynie przy bardzo małych zbiorach danych lub gdy dominującą operacją jest usuwanie elementów z początku listy.
- Drzewo BST jest lepszym wyborem przy potrzebie szybkiego wyszukiwania w dużych zbiorach danych.
- Zaletą listy jest prosta implementacja i stały czas usuwania pierwszego elementu. Wadą jest złożoność wstawiania i wyszukiwania.
- Drzewo BST oferuje logarytmiczną złożoność operacji, ale w pesymistycznym przypadku degeneruje się do listy.

Zadanie 2

Opis zadania

Na podstawie drzewa BST zbudowano wyważone drzewo AVL metodą połowienia binarnego. Elementy drzewa BST odczytano w porządku inorder, a następnie zastosowano metodę połowienia binarnego do konstrukcji zrównoważonego drzewa. Porównano wysokości obu drzew w zależności od liczby elementów.





Analiza wyników

Porównanie wysokości drzew pokazało, że standardowe drzewo BST jest zazwyczaj ponad dwa razy wyższe niż wyważone drzewo AVL. Dla 150 tysięcy elementów wysokość BST była równa 40-43, podczas gdy drzewo wyważone osiągnęło wysokość zaledwie 18. Ma to duże znaczenie, bo wysokość drzewa to w praktyce liczba przystanków, jakie komputer musi zaliczyć, by znaleźć konkretną liczbę.

Proces wyważania służy przede wszystkim zabezpieczeniu się przed najgorszym scenariuszem, czyli sytuacją, w której drzewo stałoby się długą, wąską linią przypominającą listę. Skutkiem wyważenia jest maksymalne skrócenie ścieżek dostępu do danych. Jest to szczególnie sensowne w systemach, gdzie dane raz wprowadzone są później przeszukiwane tysiące razy.

Cel i skutki wyważania drzewa

Proces wyważania drzewa służy przede wszystkim:

- Zabezpieczeniu przed degeneracją drzewa – sytuacją, w której staje się ono długą listą o złożoności O(n) zamiast O(log n).
- Maksymalnemu skróceniu ścieżek dostępu do danych – wyważone drzewo gwarantuje złożoność O(log n) dla wszystkich operacji.

Wyważanie jest szczególnie sensowne, gdy dane wstawiane są w sposób bliski posortowanemu – w takim przypadku standardowe BST degeneruje się do listy, a wyważanie przywraca optymalne własności drzewa

Wnioski

Analiza pokazała, że struktury dynamiczne dają dużą elastyczność w zarządzaniu pamięcią, bo rosną wraz z danymi. Jednak to uporządkowane struktury statyczne gwarantują stabilny, krótki czas dostępu. Ostatecznie dobór struktury to miecz obosieczny: albo poświęcamy czas na wstępne wyważanie, aby później szybko je odczytywać, albo stawiamy na szybkość zapisu kosztem późniejszego, wolniejszego wyszukiwania

Narzędzia pomiarowe

Język i środowisko programistyczne

Język: Python3
Środowisko: PyCharm (macOS)

Precyzja pomiaru czasu

Do pomiaru czasu wykonania operacji sortowania użyto funkcji *time.pref_counter*(). Jest to licznik o najwyższej dostępnej rozdzielczości w systemie, przeznaczony specjalnie do pomiarów typu benchamrk. W przeciwieństwie do standardowego *time.time*(), jest on odporny na systemowe aktualizacje czasu, co zapewnia stabilność wyników.
Jednostka miary: Sekundy[*s*]

Przetwarzania i wizualizacja danych

Biblioteka Pandas: Posłużyła do agregacji surowych wyników w strukturę tabelaryczną oraz automatycznego eksportu danych do pliku formatu .csv (w celu późniejszej weryfikacji danych).
Biblioteka Matplotlib: Wykorzystana do generowania wykresu zależności $t = f(n)$.